

Ficha de Avaliação – UFCD 10810

FICHA DE AVALIAÇÃO – UFCD 10810

Fundamentos do Desenvolvimento de Modelos Analíticos em Python

Objetivo: Avaliar a aplicação de conhecimentos em modelos analíticos supervisionados e não supervisionados, com base em datasets reais e públicos.

Os Exercícios **1 e 2 são totalmente orientados, pelo que bastará reproduzir o código**. Se quiseres podes acrescentar melhorias.

No final são apresentados dois exercícios que deverás resolver autonomamente.

Por fim, anexa todos os ficheiros ou ficheiro obtidos na resolução desta Ficha à secção correspondente no moodle.

Exercício 1 – Clustering com K-Means (Modelo Não Supervisionado)

(5-valores)

Faz o download do dataset em <https://www.kaggle.com/datasets/abdallahwagih/mall-customers-segmentation>

Contexto:

Pretende-se segmentar clientes de um centro comercial com base nas suas características de consumo, para apoiar ações de marketing direcionado.

Passos obrigatórios:

1. Carregar e explorar os dados (verificar colunas e tipos).
2. Aplicar pré-processamento:
 - a. Tratar valores ausentes, se existirem.
 - b. Normalizar as colunas numéricas.
 - c. Codificar variáveis categóricas com OneHotEncoder, se necessário.
3. Determinar o número ideal de clusters usando o ****Modelo do Cotovelo****.
4. Aplicar ****K-Means**** com o número ótimo de clusters.
5. Avaliar o clustering com o ****Coeficiente de Silhueta****.
6. Visualizar os clusters em 2D com ****PCA****.

7. Interpretar os grupos formados (ex: características dos clientes por grupo).

```
# exercicio1_kmeans_mall.py
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# 1. Carregar dados
df = pd.read_csv("Mall_Customers.csv")
print(df.info())
print(df.head())

# 2. Pré-processamento
# Verificar e remover colunas não numéricas ou pouco informativas
df = df.drop("CustomerID", axis=1)

# One-hot encoding da coluna 'Genre'
df = pd.get_dummies(df, columns=["Genre"], drop_first=True)

# Normalização
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

# 3. Modelo do Cotovelo
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title("Método do Cotovelo")
plt.xlabel("Número de clusters")
plt.ylabel("WCSS")
plt.show()

# 4. Aplicar KMeans com k=5 (por exemplo)
k = 5
kmeans = KMeans(n_clusters=k, random_state=0)
clusters = kmeans.fit_predict(scaled_data)
```

```
# 5. Avaliar com coeficiente de silhueta
score = silhouette_score(scaled_data, clusters)
print(f"Coeficiente de Silhueta: {score:.2f}")

# 6. Visualizar com PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)

plt.scatter(pca_data[:, 0], pca_data[:, 1], c=clusters, cmap='viridis')
plt.title("Clusters com PCA")
plt.xlabel("Componente 1")
plt.ylabel("Componente 2")
plt.colorbar()
plt.show()
```

Exercício 2 – Modelo Supervisionado (Classificação com Random Forest)

(5-valores)

Dataset: <https://www.kaggle.com/datasets/cherngs/heart-disease-cleveland-uci>

Contexto:

Pretende-se prever a presença de doença cardíaca a partir de dados clínicos.

Passos obrigatórios:

1. Carregar e explorar os dados.
2. Aplicar pré-processamento:
 - a. Normalizar variáveis numéricas.
 - b. Codificar variáveis categóricas com OneHotEncoder.
 - c. Verificar e remover outliers (usar **IQR** e/ou **Z-score**).
3. Separar dados em treino/teste com **stratificação**.
4. Aplicar o modelo **Random Forest** com validação cruzada.
5. Otimizar hiperparâmetros com **GridSearchCV**.
6. Avaliar o modelo com:
 - a. **Matriz de confusão**
 - b. **Acurácia, Precisão, Revocação, F1-score**
7. Comparar desempenho antes e depois da otimização
8. Prever o risco de doença para um novo paciente (valores fictícios).

```

# exercicio2_randomforest_heart.py (sem scipy)
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 1. Carregar dados
df = pd.read_csv("heart_cleveland_upload.csv")
print("Colunas disponíveis:", df.columns.tolist())
print(df.head())

# 2. Definir variáveis categóricas e numéricas com base na descrição
categorical_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'thal']
numerical_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'ca']

# 3. Remover outliers com base no IQR
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    filtro = (df[col] >= Q1 - 1.5 * IQR) & (df[col] <= Q3 + 1.5 * IQR)
    df = df[filtro]

# 4. Separar X e y
X = df[categorical_cols + numerical_cols]
y = df["condition"]

# 5. Dividir treino/teste com estratificação
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0
)

# 6. Pipeline com pré-processamento e modelo
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)

pipeline = Pipeline([
    ("pre", preprocessor),

```

```

    ("clf", RandomForestClassifier(random_state=0))
])

# 7. Validação cruzada
cv_scores = cross_val_score(pipeline, X_train, y_train, cv=5)
print(f"Acurácia média (antes da otimização): {cv_scores.mean():.2f}")

# 8. Otimização com GridSearchCV
param_grid = {
    "clf__n_estimators": [50, 100],
    "clf__max_depth": [None, 10, 20]
}

grid = GridSearchCV(pipeline, param_grid, cv=5)
grid.fit(X_train, y_train)

# 9. Avaliação do modelo
y_pred = grid.predict(X_test)
print("Matriz de Confusão:\n", confusion_matrix(y_test, y_pred))
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# 10. Previsão para novo paciente (valores médios fictícios)
novo_paciente = pd.DataFrame([
    {
        'age': 55, 'trestbps': 130, 'chol': 245, 'thalach': 150,
        'oldpeak': 1.0, 'ca': 0, 'sex': 1, 'cp': 1, 'fbs': 0,
        'restecg': 1, 'exang': 0, 'slope': 2, 'thal': 2
    }
])

pred_novo = grid.predict(novo_paciente)
print(f"\nRisco previsto para novo paciente: {'Doente' if pred_novo[0] == 1 else 'Saudável'}")

```

Exercício 3 – Modelo Supervisionado (Classificação com K-Nearest Neighbors) (5-valores)

Dataset: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>

Contexto:

Pretende-se prever se um paciente corre risco de falecer devido a insuficiência cardíaca, com base em dados clínicos.

Tarefas:

1. Carregar e explorar os dados (verificar colunas e tipos).
2. Aplicar pré-processamento:
 - Tratar valores ausentes, se existirem.
 - Normalizar variáveis numéricas.
 - Codificar variáveis categóricas com OneHotEncoder.
 - Verificar e remover outliers com **IQR**.
3. Separar os dados em treino/teste com **stratificação**.
4. Aplicar um modelo **K-Nearest Neighbors (KNN)**.
5. Avaliar o modelo com:
 - **Matriz de confusão**
 - **Acurácia, Precisão, Revocação e F1-score**
6. Otimizar o número de vizinhos com **GridSearchCV**.
7. Comparar o desempenho antes e depois da otimização.
8. Prever a condição de um novo paciente (com valores fictícios).

Exercício 4 – Modelo Não Supervisionado (Clustering com DBSCAN)

(5-valores)

Dataset: <https://archive.ics.uci.edu/dataset/292/wholesale+customers>

Contexto:

Pretende-se segmentar clientes de uma empresa grossista com base no tipo de produtos que comprou, para identificar padrões de consumo e apoiar decisões de negócio.

Tarefas:

1. Carregar e explorar os dados (verificar colunas e tipos).

2. Aplicar pré-processamento:
 - Tratar valores ausentes, se existirem.
 - Normalizar as variáveis numéricas com StandardScaler.
 - Verificar e remover outliers com **IQR**.
3. Aplicar o algoritmo **DBSCAN** para clustering.
4. Avaliar o clustering:
 - Número de clusters e de outliers identificados.
 - **Coeficiente de Silhueta**.
5. Visualizar os clusters em 2D com **PCA**.
6. Interpretar os grupos encontrados (comportamento de compra típico).